

Cours 21 — Au cœur de l'ordinateur

Semaine 21 ■ Manuel Hachette ch. 1, cours p. 12 à 19 ■ 2 × 1 h

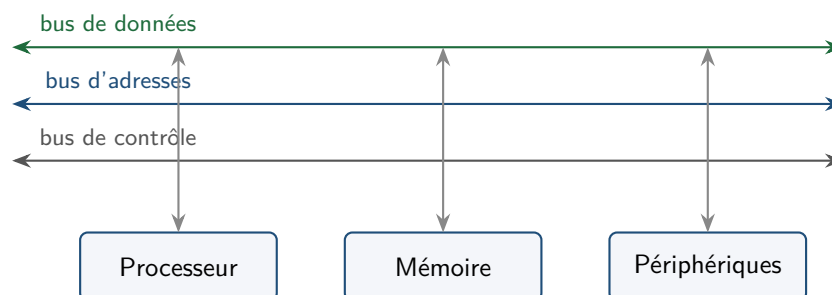
Objectifs

- Décrire l'architecture de von Neumann : processeur (UC, UAL, registres), mémoire, périphériques, les trois bus
- Expliquer le cycle d'exécution d'une instruction
- Lire et écrire un petit programme en assembleur (jeu d'instructions du manuel)

1. L'architecture d'un ordinateur (manuel § 1 et 2)

Un ordinateur est une machine de traitement de l'information. Son organisation, due à John von Neumann (1945), est la même du serveur au smartphone :

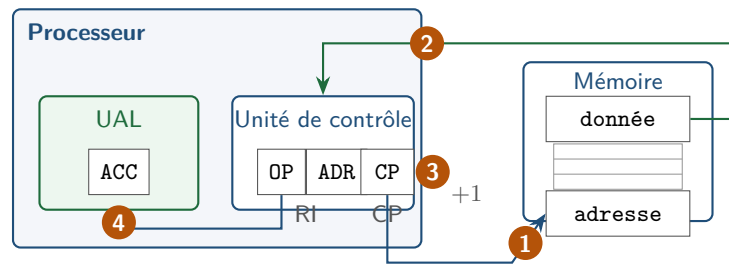
- le **processeur**, qui exécute les traitements ;
- la **mémoire**, qui stocke les données **et les programmes** — c'est l'idée révolutionnaire : un programme se charge et se remplace comme une donnée ;
- les **périphériques** pour communiquer avec l'extérieur (écran, clavier, disque, réseau) ;
- trois **bus** qui les relient : le bus **d'adresses** (quel emplacement ?), le bus de **données** (la valeur lue ou écrite), le bus de **contrôle** (coordination, signaux lecture/écriture).



La **mémoire** est un grand parking de cases numérotées : chaque emplacement a une **adresse** unique. Pour lire : on place l'adresse dans le **registre d'adresse**, la mémoire recopie le contenu dans le **registre de donnée**. La **mémoire vive** (RAM) est rapide mais s'efface à l'extinction ; la **mémoire de masse** (disque) est lente mais persistante.

Le **processeur** contient :

- l'**unité arithmétique et logique** (UAL), qui calcule (additions, soustractions, comparaisons) autour de son registre de travail : l'**accumulateur** (ACC) ;
- l'**unité de contrôle** (UC), qui enchaîne les instructions grâce à deux registres : le **compteur de programme** (CP — l'adresse de la prochaine instruction) et le **registre d'instruction** (RI — l'instruction en cours, décomposée en **code opération** OP et **opérande** ADR) ;
- une **horloge** qui cadence tout (fréquence en GHz : milliards de cycles par seconde).



Le cycle d'exécution : ① CP envoie l'adresse — ② l'instruction arrive dans RI — ③ CP +1 — ④ OP décodé part vers l'UAL.

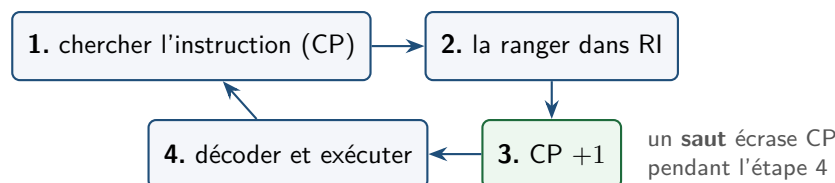
Unités à connaître (notes du manuel p. 12) : un **octet** = 8 bits ; méga = million, giga = milliard, téra = mille milliards. La mémoire se mesure en Go/To, la vitesse du processeur en GHz.

2. Le cycle d'exécution (manuel § 3)

L'unité de contrôle répète sans fin :

1. l'adresse contenue dans **CP** est envoyée à la mémoire ;
2. l'instruction lue arrive dans **RI** ;
3. **CP est incrémenté** de 1 (prêt pour la suivante) ;
4. l'instruction est **décodée** : le code opération part vers l'UAL, qui exécute (calcul, lecture/écriture mémoire, ou saut).

Cas particulier des **sauts** : l'instruction écrase CP avec une autre adresse — la « prochaine instruction » n'est plus la suivante.



3. Programmer en assembleur (manuel § 4)

En mémoire, une instruction est un mot binaire. L'**assembleur** est sa notation lisible. Le jeu d'instructions du manuel :

instruction	effet
LOAD adr / LOAD #n	mémoire[adr] (ou le nombre n) → ACC
STORE adr	ACC → mémoire[adr]
ADD adr / ADD #n	ACC + valeur → ACC (idem SUB, MUL, DIV)
CMP adr / CMP #n	compare ACC à la valeur : -1, 0 ou 1 → ACC
JMP adr	saut : adr → CP
JMPZ / JMPN / JMPP adr	saut si ACC nul / négatif / positif
HALT	arrêter le programme

L'addition du manuel (p. 15) — trois instructions pour $c = a + b$:

```

0  LOAD 100    ; lire la mémoire à l'adresse 100
1  ADD 101     ; ajouter le contenu de l'adresse 101
2  STORE 102   ; stocker le résultat à l'adresse 102
3  HALT       ; fin du programme
  
```

Pour la lisibilité, le manuel introduit les **définitions** (`def X 100` : nommer une adresse) et les **étiquettes** (`boucle:` : nommer une ligne pour les sauts).

Le maximum de X et Y (p. 16) — un **if/else** devient deux sauts :

```
def X 100
def Y 101
def R 102
LOAD X      ; charger X
CMP Y       ; comparer : -1 si X < Y
JMPN neg    ; aller à 'neg' si X < Y
LOAD X      ; cas X >= Y
JMP suite
neg: LOAD Y  ; cas X < Y
suite: STORE R
HALT
```

Et une **boucle** n'est qu'un saut en arrière (`JMPP boucle`) : le programme 2^N du manuel (p. 17) multiplie R par 2 et décrémente N tant que N est positif. Une boucle doit toujours contenir un test qui permet d'en sortir — sinon le programme « boucle » indéfiniment.

Tout Python se réduit à cela : charger, calculer, ranger, sauter. Un **while** = une étiquette + un saut conditionnel ; un **if/else** = deux sauts. C'est l'objet du TP sur le simulateur du manuel.

4. Puissance et limites (manuel § 6, 7 et p. 8)

Un peu d'histoire (frise p. 8) : boulier, Pascaline (1642), métier Jacquard (1804), machine analytique de Babbage (1830), article fondateur de Turing (1936), ENIAC (1945), architecture de von Neumann (1945), transistor (1947), circuit intégré (1958), microprocesseur Intel 4004 (1970), Apple I (1976), IBM PC (1981).

La **loi de Moore** (1965) : le nombre de transistors double tous les 18 mois — d'où 40 ans de croissance exponentielle, aujourd'hui relayée par les architectures **multi-processeurs** (mémoire partagée ou interconnectée — détails en semaine 22).

Les ordinateurs peuvent-ils tout calculer ? (§ 7) Non : il existe des fonctions **non calculables**, et savoir si un programme « boucle » est **indécidable**. Mais tout ce qui est calculable l'est par un ordinateur (thèse de Church) — la différence ne se joue qu'en vitesse.

À retenir

- Von Neumann : processeur (UC + UAL + registres ACC, RI, CP), mémoire (données et programme), périphériques, 3 bus (adresses, données, contrôle).
- Cycle : CP → mémoire → RI, CP+1, décodage, exécution par l'UAL ; un saut écrase CP.
- Assembleur : LOAD, STORE, ADD/SUB, CMP, JMP/JMPZ/JMPN/JMPP, HALT ; boucle = saut en arrière avec test de sortie.
- Loi de Moore, multi-processeurs ; certaines questions sont incalculables (indécidables).